Learning Structural Representations for Recipe Generation and Food Retrieval

Hao Wang, Guosheng Lin, Steven C. H. Hoi, *Fellow, IEEE*, and Chunyan Miao

Abstract—Food is significant to human daily life. In this paper, we are interested in learning structural representations for lengthy recipes, that can benefit the recipe generation and food cross-modal retrieval tasks. Different from the common vision-language data, here the food images contain mixed ingredients and target recipes are lengthy paragraphs, where we do not have annotations on structure information. To address the above limitations, we propose a novel method to unsupervisedly learn the sentence-level tree structures for the cooking recipes. Our approach brings together several novel ideas in a systematic framework: (1) exploiting an unsupervised learning approach to obtain the sentence-level tree structure labels before training; (2) generating trees of target recipes from images with the supervision of tree structure labels learned from (1); and (3) integrating the learned tree structures into the recipe generation and food cross-modal retrieval procedure. Our proposed model can produce good-quality sentence-level tree structures and coherent recipes. We achieve the state-of-the-art recipe generation and food cross-modal retrieval performance on the benchmark Recipe1M dataset.

Index Terms—Text generation, vision-and-language

1 Introduction

Food-related research with the newly evolved deep learning-based techniques is becoming a popular topic, as food is essential to human life. In this paper, we investigate the tasks of recipe generation [1] and food cross-modal retrieval [2], where we enhance their performance with the unsupervisedly learned recipe tree structures. To be specific, in the recipe generation task, we aim to generate the corresponding and coherent cooking instructions for the given food images with a language decoder. In the food cross-modal retrieval task, the goal is to retrieve the matched food images given recipes as the query, and vice versa.

Both recipe generation and food cross-modal retrieval models aim to produce the corresponding recipes from given images. Since direct transforms from image to text would be difficult [3], many recent methods [4], [5], [6] for image captioning and cross-modal retrieval adopt the detected object features to enhance image features. Moreover, learning scene graphs from the detected objects can boost the cross-modal task performance further [7], [8]. This depicts the significance of using the graph information for the recipe generation and food cross-modal retrieval models.

In the food dataset Recipe1M [2], there have food images and the paired text annotations including ingredients and cooking instructions, where the food images are static and contain all the mixed ingredients. Generally, the task settings of the recipe generation and food cross-modal retrieval are almost the same as those of image captioning and general cross-modal retrieval. However, there still exist two differences between them: (i) the given text length and (ii) the annotations on the data structural information.

First, most popular image-text cross-modal datasets, such as Flickr [9] and MS-COCO [10] datasets, only have one sentence per caption. By contrast, cooking instructions are paragraphs, containing multiple sentences to represent the cooking process, which cannot be fully shown in a single food image. Therefore, generating lengthy recipes with traditional image captioning models [4], [5], [7] may hardly capture the whole cooking procedure. Second, the lack of structural information labeling is another challenge in our researched food cross-modal tasks. For example, MS-COCO [10] has precise bounding box annotations in images, giving scene graph information for the caption generation or the retrieval process. This structural information makes it easier to represent the inner relationships of the image or text data. While in food images, all the ingredients are mixed when cooked, making it difficult to obtain the detection labeling in the food images. Moreover, there are no existing works attempting to learn the structure representations for the cooking recipes. To address this limitation, we propose to learn the recipe tree structures in an unsupervised manner, integrate the learned trees into the recipe generation and food cross-modal retrieval tasks and improve their performance.

- *Hao Wang, Guosheng Lin, and Chunyan Miao are with the School of Computer Science and Engineering, Nanyang Technological University, Singapore 639798. E-mail: hao005@e.ntu.edu.sg, {gslin, ascymiao}@ntu.edu.sg.*
- *Steven C. H. Hoi is with the Singapore Management University, Singapore 188065. E-mail: chhoi@smu.edu.sg.*

Manuscript received 28 December 2020; revised 24 March 2022; accepted 24 May 2022. Date of publication 10 June 2022; date of current version 3 February 2023. This work was supported in part by the National Research Foundation (NRF), Singapore through its AI Singapore Programme under Grant AISG-GC-2019-003 and through its NRF Investigatorship Programme under Grant NRFNRFI05-2019-0002, in part by the Singapore Ministry of Health through its National Innovation Challenge on Active and Confident Ageing under Grant MOH/NIC/HAIG03/2017, in part by the National Research Foundation, Singapore through its AI Singapore Programme under Grant AISG-RP-2018-003, and in part by the MOE AcRF Tier-1 Research under Grant RG95/20.
(Corresponding authors: Chunyan Miao and Guosheng Lin.)
Recommended for acceptance by Z. Akata.
Digital Object Identifier no. 10.1109/TPAMI.2022.3181294

0162-8828 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Limited to: CENTRE FOR DEVELOPMENT OF ADVANCED COMPUTING - CDAC - Thiruvananthapuram & Kochi. Downloaded on September 06,2024 at 08:11:19 UTC from IEEE Xp

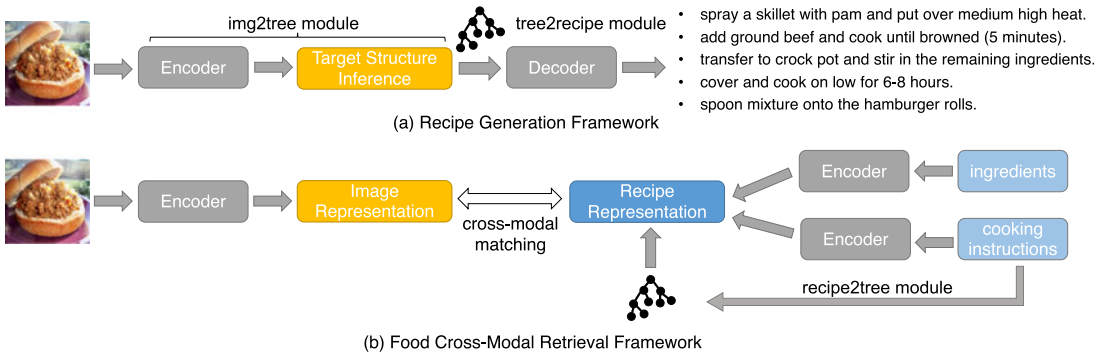


Fig. 1. The demonstration of our proposed frameworks. In the recipe generation model (a), we use the img2tree module to infer the recipe tree structures, which are encoded with the graph attention networks to give tree embeddings in the tree2recipe module. Then we generate the recipes. In the food cross-modal retrieval framework (b), we incorporate the tree structures from the recipe2tree module into the original recipe features, which boosts food cross-modal matching performance.

Specifically, benefiting from the recent advances in the language parsing, some research works, such as ON-LSTM [11], utilize an unsupervised way to produce word-level parsing trees of sentences and achieve good results. Inspired by that, we propose a novel recipe2tree module, where we extend the ON-LSTM architecture to do sentence-level tree structure generation. We propose to train the extended ON-LSTM with the Quick Thoughts manner [12], to capture the order information inside recipes. By doing so, we get the recipe tree structure labels.

We then apply the obtained recipe structure information on the recipe generation and food cross-modal retrieval tasks. For the recipe generation, we propose a novel framework named **S**tructure-aware **G**eneration **N**etwork (SGN) to integrate the tree structure information into the training and inference phases, as shown in Fig. 1a. SGN is implemented to add a target structure inference module on the recipe generation process. Specifically, we propose to use a RNN to generate the recipe tree structures from food images. Based on the generated trees, we adopt the graph attention networks (GAT) to embed the trees, in an attempt to giving the model more guidance when generating recipes. With the tree structure embeddings, we make the generated recipes remain long-length as the ground truth, and improve the generation performance considerably.

We further demonstrate the efficacy of our unsupervisedly-learned recipe tree structures in the food cross-modal retrieval task, where we incorporate the tree representations into the recipe features, as shown in Fig. 1b. Specifically, we first produce the tree structures from the cooking instructions with our proposed recipe2tree module. Then we also use the GAT to give the tree features, which are adopted to fuse with the recipe representations. We conduct the cross-modal matching between the enhanced image and recipe representations.

Our contributions can be summarized as:

- We propose a recipe2tree module to capture latent sentence-level tree structures for recipes, which are learned through an unsupervised approach. The obtained tree structures are adopted to supervise the following img2tree module.
- We propose to use the img2tree module to generate the recipe tree structures from food images, where we use a RNN for conditional tree generation.
- We propose to utilize the tree2recipe module, which encodes the inferred tree structures. It is implemented with graph attention networks, and boosts the recipe generation performance.
- We show the tree structures learned in the recipe2tree module can also help on improving the cross-modal retrieval performance.

Fig. 1 shows the concise demonstration on our proposed frameworks for the recipe generation and food cross-modal retrieval tasks. We have conducted extensive experiments to evaluate the recipe generation and food retrieval performance, showing our proposed method outperforms state-of-the-art baselines on Recipe1M dataset [2]. We also present ablation studies as well as the qualitative results of the recipe generation and food cross-modal retrieval results.

Our preliminary research has been published in [13]. The code is publicly available.[1]

2 Related Work

2.1 Image Captioning

Image captioning task is defined as generating the corresponding text descriptions from images. Based on MS-COCO dataset [10], most existing image captioning techniques adopt deep learning-based model. One popular approach is Encoder-Decoder architecture [5], [14], [15], [16], where a CNN is used to obtain the image features along with object detection, then a language model is used to convert the image features into text.

Since image features are fed only at the beginning stage of generation process, the language model may face vanishing gradient problem [17]. Therefore, image captioning model is facing challenges in long sentence generation [14]. To enhance text generation process, [4], [7] involve scene graph into the framework. However, scene graph generation rely heavily on object bounding box labeling, which is provided by MS-COCO dataset. When we shift to some other datasets without rich annotation, we can hardly obtain the graph structure information of the target text. Meanwhile, crowdsourcing annotation is high-cost and may not be reliable. Therefore, we propose to produce tree structures for paragraphs unsupervisedly, helping the recipe generation task in Recipe1M dataset [2].

1. https://github.com/hwang1996/SGN

imited to: CENTRE FOR DEVELOPMENT OF ADVANCED COMPUTING - CDAC - Thiruvananthapuram & Kochi. Downloaded on September 06,2024 at 08:11:19 UTC from IEEE Xp

2.2 Multimodal Food Computing

Food computing [18] has raised great interest recently, it targets applying computational approaches for analyzing multimodal food data for recognition [19], retrieval [2], [20], [21] and generation [1] of food. In this paper, we choose Recipe1M dataset [2] to validate our proposed method on recipe generation and food cross-modal retrieval task.

Recipe generation is a challenging task, it is mainly because that recipes (cooking instructions) contain multiple sentences. Salvador *et al.* [1] adopt transformer to generate lengthy recipes, but they fail to consider the holistic recipe structure before generation, hence their generated recipes may miss some steps. In contrast, our proposed method allows the model to predict the recipe tree structures first, and then give better generation results. Food cross-modal retrieval targets retrieving matched items given one food image or recipe. Prior works [2], [20], [22], [23] mainly aim to align the cross-modal embeddings in the common space, we improve the retrieval baseline results by enhancing the recipe representations with learned tree structures.

2.3 Image-to-Text Retrieval

The image-to-text retrieval task is to retrieve the corresponding image given the text, and vice versa. Prevailing methods [2], [21], [24], [25] adopt the deep neural networks to give the image and text features respectively, and use the metric learning to map the cross-modal features into a common space, such that the alignment between the text and images can be achieved. Specifically, Vo *et al.* [24] utilize the image plus some text to retrieve the images with certain language attributes. They propose to combine image and text through residual connection and produce the image-text joint features to do the retrieval task. Chen *et al.* [25] conduct experiments with the same setting as [24], where they use a composite transformer to plug in a CNN and then selectively preserve and transform the visual features conditioned on language semantics. In the domain of food cross-modal retrieval, Salvador *et al.* [2] aim to learn joint embeddings (JE) for images and recipes, where they adopt cosine loss to align image-recipe pairs and classification loss to regularize the learning. Zhu *et al.* [26] use two-level ranking loss at embedding and image spaces in $\mathbf{R^2GAN}$. Wang *et al.* [20] introduce the translation consistency component to allow feature distributions from different modalities to be similar.

2.4 Language Parsing

Parsing is served as one effective language analysis tool, it can output the tree structure of a string of symbols. Generally, language parsing is divided into word-level and sentence-level parsing. Word-level parsing is also known as grammar induction, which aims at learning the syntactic tree structure from corpora data. Some of the research works use a supervised way to predict the corresponding latent tree structure given a sentence [27], [28]. However, precise parser annotation is hard to obtain. [11], [29], [30] explored to learn the latent structure without the expert-labeled data. Especially, Shen *et al.* [11] propose to use ON-LSTM, which equips the LSTM architecture with an inductive bias towards learning latent tree structures. They train the model with normal language modeling way, at the same time they can get the parsing output induced by the model.

Sentence-level parsing is used to identify the elementary discourse units in a text, and it brings some benefits to discourse analysis. Many recent works attempted to use complex model with labeled data to achieve the goal [31], [32]. Here we extend ON-LSTM [11] for unsupervised sentence-level parsing, which is trained using quick thoughts [12].

2.5 Graph Generation

Graph is the natural and fundamental data structure in many fields, such as social networks and biology, and a tree is an undirected graph. The basic idea of graph generation model is to make auto-regressive decisions during graph generation. For example, Li *et al.* [33] add graph nodes and edges sequentially with auto-regressive models. The tree generation approach we use is similar with GraphRNN [34]. They [34] first map the graph to sequence under random ordering, then use edge-level and graph-level RNN to update the adjacency vector. While in our tree generation method, we generate the tree conditioned on food images, and the node ordering is fixed according to hierarchy, which releases the complexity of the sampling space.

3 Method

Here we investigate two research tasks of 1) food recipe generation from images and 2) food cross-modal retrieval. We give the unified training flow demonstration for these two tasks in Fig. 2. In the following sections, we present our proposed models for recipe generation and food cross-modal retrieval, whose frameworks are shown in Figs. 3 and 5 respectively.

3.1 Overview

We show the concise training flows for recipe generation and food cross-modal retrieval models in Fig. 2, where we first use image encoders to extract visual features and use the recipe2tree module to give recipe tree structures. Specifically, adopting the generated pseudo structures of the recipe2tree module is the core component of recipe generation and cross-modal retrieval models, which is shared by both models. We learn recipe sentence-level tree structures in an unsupervised manner at the recipe2tree module. Then in the recipe generation model, the generated pseudo trees are used to supervise the tree generation from images; in the cross-modal retrieval model, the tree structures are adopted to enhance the recipe language features.

Technically, in the recipe2tree module we propose to use the hierarchical ON-LSTM [11] to encode the cooking instructions and train the ON-LSTM with quick thoughts approach [12]. Then we can obtain the latent tree structures of cooking instructions, which are used as the pseudo labels to supervise the training of the img2tree module.

During the recipe generation training phase, as shown in Figs. 2 and 3, we input food images and ingredients to our proposed model. We try two different language models to encode ingredients, i.e., non-pretrained and pretrained model, to get the ingredient features F_{ing}. In the non-pretrained model training, we use one word embedding layer [1] to give F_{ing}. Besides, we adopt BERT [35] for ingredient embedding, which is one of the state-of-the-art NLP pretrained models. In the image embedding branch, we adopt a

imited to: CENTRE FOR DEVELOPMENT OF ADVANCED COMPUTING - CDAC - Thiruvananthapuram & Kochi. Downloaded on September 06,2024 at 08:11:19 UTC from IEEE Xp

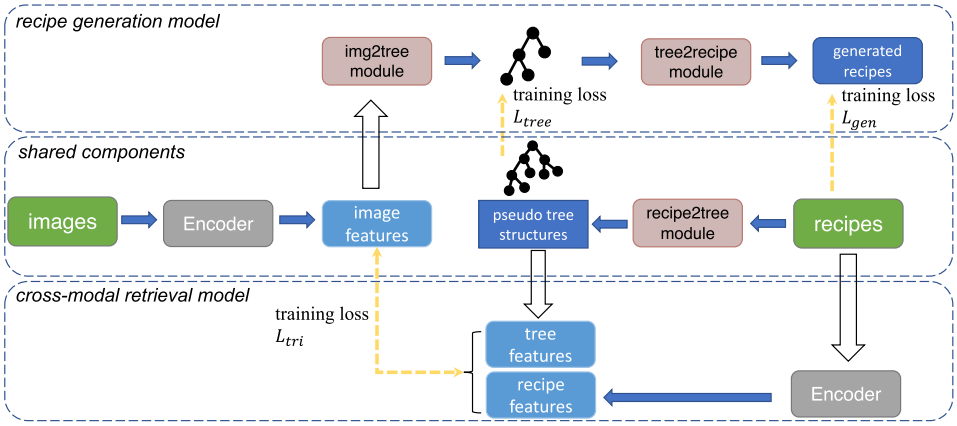


Fig. 2. The concise training flow of our proposed models for recipe generation and food cross-modal retrieval, which are trained individually. In the shared components, we extract the food image features and use the recipe2tree module to produce pseudo recipe tree structures. In the recipe generation model, we generate trees with the img2tree module that is trained with L_{tree}. The predicted tree structures from the img2tree module are used to generate the recipes in the tree2recipe module, which is supervised by L_{gen}. In the cross-modal retrieval model, we concatenate the tree features and recipe features. The concatenated features are trained to match with the image features, which is supervised by L_{tri}.

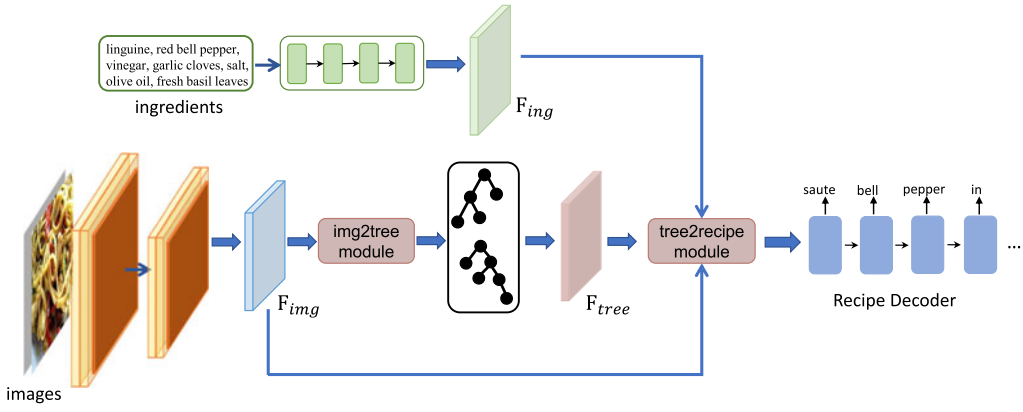


Fig. 3. Our proposed framework for recipe generation. The ingredients and food images are embedded by a pretrained language model and CNN respectively to produce the output features F_{ing} and F_{img}. Before language generation, we first infer the tree structure of target cooking instructions. To do so, we utilize the img2tree module, where a RNN produces the nodes and edge links step-by-step based on F_{img}. Then in tree2recipe module, we adopt graph attention networks (GAT) to encode the generated tree adjacency matrix, and get tree embedding F_{tree}. We combine F_{ing}, F_{img} and F_{tree} to construct a final embedding for recipe generation, which is performed using a transformer.

CNN to encode the food images and get the image features F_{img}. Based on F_{img}, we generate the sentence-level tree structures and make them align with the pseudo labels produced by the recipe2tree module. Specifically, we transform the tree structures to a 1-dimensional adjacency sequence for RNN to generate, where the RNN's initial state is image feature F_{img}. To incorporate the generated tree structure into the recipe generation process, we get the tree embedding F_{tree} with graph attention networks (GAT) [36], and concatenate it with the image features F_{img} and ingredient features F_{ing}. We then generate the recipes conditioned on the concatenated features of $\langle F_{tree}, F_{img}, F_{ing}\rangle$ with a transformer [37].

Our proposed recipe generation framework is optimized over two objectives: to generate reasonable recipes given the food images and ingredients; and to produce the sentence-level tree structures of target recipes. The overall objective is given as

$$L = \lambda_1 L_{gen} + \lambda_2 L_{tree}, \tag{1}$$

where λ_1 and λ_2 are trade-off parameters. L_{gen} controls the recipe generation training with the input of $\langle F_{tree}, F_{img}, F_{ing}\rangle$, and outputs the probabilities of word tokens. L_{tree} is the tree generation loss, supervising the img2tree module to generate trees from images.

As shown in Figs. 2 and 5, we propose to incorporate the latent trees into the food cross-modal retrieval task to further demonstrate the usefulness of our unsupervisedly-learned tree structures and boost the retrieval performance. In this task, given a food image, we want to retrieve the corresponding cooking recipe including the ingredients and the cooking instructions, or vice versa. To this end, we also adopt a CNN to give image representations F_{img} and a language encoder to give the ingredient and cooking instruction features F_{ing} and F_{ins}. We adopt the GAT to encode the sentence-level tree structures and obtain the tree representations F_{tree}. The recipe embeddings F_{rec} are constructed by the concatenation of $\langle F_{tree}, F_{ins}, F_{ing}\rangle$. We use triplet loss L_{tri} to align image and recipe representations F_{img} and F_{rec} in the feature space and learn a joint embedding for cross-modal matching.

3.2 ON-LSTM Revisit

Ordered Neurons LSTM (ON-LSTM) [11] is proposed to infer the underlying tree-like structure of language while

imited to: CENTRE FOR DEVELOPMENT OF ADVANCED COMPUTING - CDAC - Thiruvananthapuram & Kochi. Downloaded on September 06,2024 at 08:11:19 UTC from IEEE Xp

learning the word representation. It can achieve good performance in unsupervised parsing task. ON-LSTM is constructed based on the intuition that each node in the tree can be represented by a set of neurons the hidden states of recurrent neural networks. To this end, ordered neuron is an inductive bias, where high-ranking neurons store long-term information, while low-ranking neurons contain short-term information that can be rapidly forgotten. Instead of acting independently on each neuron, the gates of ON-LSTM are dependent on the others by enforcing the order in which neurons should be updated. Technically, Shen *et al.* [11] define the split point d between two segments. d_t^f and d_t^i represent the hierarchy of the previous hidden states h_{t-1} and that of the current input token x_t respectively, which can be formulated as

$$d_t^f = \text{softmax}(W_{\tilde{f}} x_t + U_{\tilde{f}} h_{t-1} + b_{\tilde{f}}), \tag{2}$$

$$d_t^i = \text{softmax}(W_{\tilde{i}} x_t + U_{\tilde{i}} h_{t-1} + b_{\tilde{i}}), \tag{3}$$

where $\tilde{f}$ and $\tilde{i}$ are defined by the ON-LSTM as the master forget gate and the master input gate. W, U and b are the learnable weights of ON-LSTM. As stated in [11], the information stored in the first d_t^f neurons of the previous cell state will be completely erased, and a large d_t^i means that the current input x_t contains long-term information that needs to be preserved for several time steps. The model weights are updated based on the predicted $\tilde{f}$ and $\tilde{i}$.

The ON-LSTM model is trained through word-level language modeling, where given one token in the document they predict the next token. With the trained ON-LSTM, Shen *et al.* attempt to do the unsupervised constituency parsing. At each time step, they compute an estimate of d_t^f

$$\hat{d}_t^f = \mathbb{E}\left[d_t^f\right] = \sum_{k=1}^{D_m} k p_f(d_t = k), \tag{4}$$

where p_f denotes the probability distribution over split points associated to the master forget gate and D_m is the size of the hidden state. Given $\hat{d}_t^f$, the top-down greedy parsing algorithm [30] is used for unsupervised constituency parsing. As described in [11], for the first $\hat{d}_i^f$, they split the sentence into constituents $((x_{<i}), (x_i, (x_{>i})))$. Then, they recursively repeat this operation for constituents $(x_{<i})$ and $(x_{>i})$, until each constituent contains only one word.

Therefore, ON-LSTM is able to discern a hierarchy between words based on the model neurons. However, ON-LSTM is originally trained by language modeling way and learns the word-level order information. To unsupervisedly produce sentence-level tree structure, we extend ON-LSTM in the recipe2tree module.

3.3 Recipe2tree Module

In this module, we propose to learn a hierarchical ON-LSTM, i.e., word-level and sentence-level ON-LSTM. Specifically, in the word-level ON-LSTM, we input the cooking recipe word tokens and use the output features as the sentence embeddings. The sentence embeddings will be fed into the sentence-level ON-LSTM for end-to-end training.

Since the original training way [11], such as language modeling or seq2seq [38] word prediction training, cannot be used in sentence representation learning, we incorporate the idea of quick thoughts (QT) [12] to supervise the hierarchical ON-LSTM training. The general objective of QT is a discriminative approximation where the model attempts to identify the embedding of a correct target sentence given a set of sentence candidates. In other words, instead of predicting *what is the next* in language modeling, we predict *which is the next* in QT training to capture the order information inside recipes. Technically, for each recipe data, we select first $N-1$ of the cooking instruction sentences as context, i.e., $S_{ctxt} = \{s_1, \ldots, s_{N-1}\}$. Then sentence s_N turns out to be the correct next one. Besides, we randomly select K sentences along with the correct sentence s_N from each recipe, to construct candidate sentence set $S_{cand} = \{s_N, s_i, \ldots, s_k\}$. The candidate sentence features $g(S_{cand})$ are generated by the word-level ON-LSTM, and the context embeddings $f(S_{ctxt})$ are obtained from the sentence-level ON-LSTM. The computation of probability is given by

$$p(s_{\text{cand}}|S_{\text{ctxt}}, S_{\text{cand}}) = \frac{\exp[c(f(S_{\text{ctxt}}), g(s_{\text{cand}}))]}{\sum_{s' \in S_{\text{cand}}} \exp[c(f(S_{\text{ctxt}}), g(s'))]}, \tag{5}$$

where c is an inner product, to avoid the model learning poor sentence encoders and a rich classifier. Minimizing the number of parameters in the classifier encourages the encoders to learn disentangled and useful representations [12]. The training objective maximizes the probability of identifying the correct next sentences for each training recipe data D

$$\sum_{s \in D} \log p(s|S_{\text{ctxt}}, S_{\text{cand}}). \tag{6}$$

We adopt the learned sentence-level ON-LSTM to give the neuron ranking for the cooking instruction sentences, which can be converted to the recipe tree structures T through the top-down greedy parsing algorithm [30]. T are adopted as the pseudo labels to supervise the training of img2tree module.

3.4 Recipe Generation

3.4.1 Img2tree Module

In the img2tree module, we aim to generate the tree structures from food images. Tree structure has hierarchical nature, in other words, "parent" node is always one step higher in the hierarchy than "child" nodes. Given the properties, we first represent the trees as sequence under the hierarchical ordering. Then, we use an auto-regressive model to model the sequence, meaning that the edges between subsequent nodes are dependent on the previous "parent" node. Besides, in Recipe1M dataset, the longest cooking instructions have 19 sentences. Therefore, the sentence-level parsing trees have limited node numbers, which avoids the model generating too long or complex sequence.

In Fig. 3, we specify our tree generation approach. The generation process is conditioned on the food images. According to the hierarchical ordering, we first map the tree structure to the adjacency matrix, which denotes the links between nodes by 0 or 1. Then the lower triangular part of the adjacency matrix will be converted to a vector $V \in \mathbb{R}^{n \times 1}$, where each element $V_i \in \{0, 1\}^i, i \in \{1, \ldots, n\}$. Since edges in tree structure are undirected, V can determine a unique tree T.

imited to: CENTRE FOR DEVELOPMENT OF ADVANCED COMPUTING - CDAC - Thiruvananthapuram & Kochi. Downloaded on September 06,2024 at 08:11:19 UTC from IEEE Xp

Here the tree generation model is built based on the food images, capturing how previous nodes are interconnected and how following nodes construct edges linking previous nodes. Hence, we adopt Recurrent Neural Networks (RNN) to model the predefined sequence V. We use the image encoded features F_{img} as the initialization of RNN hidden state, and the state-transition function h and the output function y are formulated as

$$h_0 = \mathrm{F}_{img}, h_i = f_{trans}(h_{i-1}, V_{i-1}), \tag{7}$$

$$y_i = f_{out}(h_i), \tag{8}$$

where h_i is conditioned on the previous generated $i-1$ nodes, y_i outputs the probabilities of next node's adjacency vector.

The tree generation objective function is

$$p(V) = \prod_{i=1}^{n} p(V_i | V_1, \ldots, V_{i-1}), \tag{9}$$

$$L_{tree} = \sum_{V \in D} \log p(V), \tag{10}$$

where $p(V)$ is the product of conditional distributions over the elements, D denotes all the training data.

3.4.2 Tree2recipe Module

In the tree2recipe module, we utilize graph attention networks (GAT) [36] to encode the generated trees. The input of GAT is the generated sentence-level tree adjacency matrix A and its node features. Since the sentence features are not available during recipe generation, we produce node features with a linear transformation W, which is applied on the adjacency matrix A. We then perform attention mechanism on the between connected nodes (z_i, z_j) and compute the attention coefficients

$$e_{ij} = (\mathrm{W}z_i)(\mathrm{W}z_j)^T, \tag{11}$$

where e_{ij} measures the importance of node j's features to node i, the attention coefficients are computed by the matrix multiplication.

It is notable that different from most attention mechanism, where every node attends on every other node, GAT only allows each node to attend on its neighbour nodes. The underlying reason is that doing global attention fails to consider the property of tree structure, that each node has limited links to others. While the local attention mechanism used in GAT preserves the structural information well. We can formulate the final attentional score as

$$\alpha_{ij} = \mathrm{softmax}_j(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in \mathrm{N}_i} \exp(e_{ik})}, \tag{12}$$

where N_i is the neighborhood of node i, the output score is normalized through the softmax function. Similar with [37], GAT employs multi-head attention and averaging to stabilize the learning process. We get the tree features by the product of the attentional scores and the node features, and we perform nonlinear activation σ on the output to get the final features

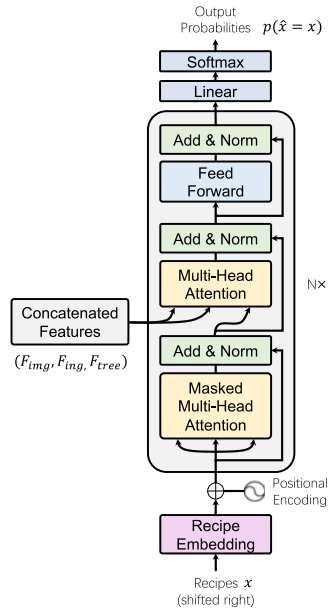


Fig. 4. The demonstration of the transformer training for the recipe generation. The concatenated features are composed of the image features F_{img}, ingredient features F_{ing} and tree structure representations F_{tree}. In the training phase, we take the ground truth recipes x as the input. The predicted token and the output probability are denoted as $\hat{x}$ and $p(\hat{x} = x)$ respectively. We set the transformer layer number $N = 16$.

$$\mathrm{F}_{tree} = \sigma\left(\sum_{j \in \mathrm{N}_i} \alpha_{ij} \mathrm{W} z_j\right). \tag{13}$$

3.4.3 Recipe Generation From Images

The demonstration of the transformer [37] structure for language generation is presented in Fig. 4. We adopt a 16-layer transformer [37] for recipe generation, which is the same setting as [1]. We use the teacher forcing training strategy, where we feed the previous ground truth word $x^{(i-1)}$ into the model and let the model generate the next word token $\hat{x}^{(i)}$ in the training phase. In the transformer attention mechanism [37], we have query Q, key K and value V, the attentional output can be computed as

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \tag{14}$$

where d_k denotes the dimension of K. Here in the Multi-Head Attention module of Fig. 4, we use the concatenated features of previously obtained F_{img}, F_{ing} and F_{tree} as K and V, and the processed recipe embeddings are used as the Q.

The training objective of the recipe generation is to maximize the following objective:

$$L_{gen} = \sum_{i=0}^{M} \log p(\hat{x}^{(i)} = x^{(i)}), \tag{15}$$

where L_{gen} is the recipe generation loss, and M is the maximum sentence generation length, $x^{(i)}$ and $\hat{x}^{(i)}$ denote the ground truth and generated tokens respectively. In the inference phase, the transformer decoder outputs $\hat{x}^{(i)}$ one by one.

3.5 Food Cross-Modal Retrieval

The training framework for the food cross-modal retrieval task is shown in Fig. 5. We follow the same food cross-modal

imited to: CENTRE FOR DEVELOPMENT OF ADVANCED COMPUTING - CDAC - Thiruvananthapuram & Kochi. Downloaded on September 06,2024 at 08:11:19 UTC from IEEE Xp

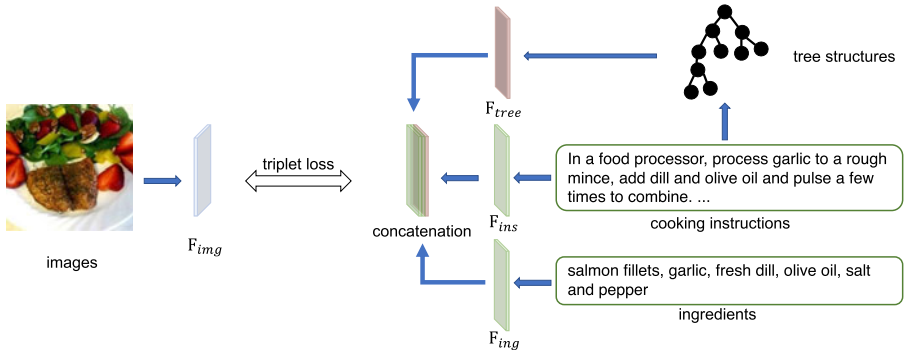


Fig. 5. The training flow of food cross-modal retrieval. We first produce the sentence-level tree structures from the cooking instructions, where we use the sentence features as the node features. The tree structure, cooking instruction and ingredient features are denoted as F_{tree}, F_{ins} and F_{ing} respectively. We use the concatenation of F_{tree}, F_{ins} and F_{ing} as the recipe features. The triplet loss is adopted to learn the similarity between the recipe features F_{rec} and image features F_{img}.

retrieval setting as [2], [20], [21], where given a food image we aim to find the corresponding cooking recipe, and vice versa. To this end, we first obtain the feature representations of food images and recipes respectively, then we learn the similarity between the food images and cooking recipes through the triplet loss. Technically, we get the food image representations F_{img} from the output of CNN directly, and get the recipe representations F_{rec} from the concatenation of the ingredient features F_{ing}, instruction features F_{ins} and recipe tree structure representation F_{tree}. We project F_{img} and F_{rec} into a common space and align them to realize cross-modal retrieval.

3.5.1 Ingredient Embedding

Regarding the ingredient embedding process, we implement LSTM and transformer [37] respectively as the encoder to produce ingredient features F_{ing}. We first follow [2], [20] to use the ingredient-level word2vec representations $y \in \{y_1, \ldots, y_n\}$ for ingredient token representations. For example, *ground ginger* is regarded as a single word vector, instead of two separate word vectors of *ground* and *ginger*. Then the processed word2vec representations y are fed into the ingredient encoder, where we experiment with both the bidirectional LSTM and the transformer. Specifically, we follow [21], [37] and implement the self-attention mechanism on the LSTM to boost the performance for the fair comparison with the transformer. The transformer is constructed with 4 layers. We use the final state output of the ingredient encoder as the ingredient features F_{ing}.

3.5.2 Cooking Instruction Embedding

To obtain the cooking instruction features F_{ins}, we also follow previous practice [2], [20] to extract the sentence features for fair comparisons. We first obtain the fixed-length representation $r \in \{r_1, \ldots, r_n\}$ for each cooking instruction sentence with the skip-thoughts [39] technique. Then we feed r into the instruction encoder to get the sequence embeddings F_{ins} for cooking instructions. Here we also experiment with both the LSTM and the transformer as the instruction encoder, where the LSTM is enhanced with the self-attention mechanism [37] and the transformer has 4 layers.

3.5.3 Tree Structure Embedding

We further newly introduce the sentence-level tree structure representations to improve the cooking instruction features. To this end, we produce the tree structures T from the given cooking instruction sentences, which are generated by the recipe2tree module introduced in Section 3.3. T is converted into the adjacency matrix A, such that we can use GAT to emb sentence-level trees T and obtain structure representations F_{tree} for cooking instructions. It is notable that here tree representation F_{tree} construction method is different from that in Section 3.4.2, where the cooking instruction sentence representations are not available during generation phase, in the retrieval setting we can use sentence embeddings as node features. Technically, we denote the cooking instruction sentence embeddings from the skip-thoughts [39] as $r \in \{r_1, \ldots, r_n\}$. The child node representations are set as r, and the parent node representation are set as the mean of its child node representations. Hence the node representations with the sentence embeddings can be denoted as f_{node}^{sen}. Moreover, since the learned tree structures have the hierarchical property, we incorporate additional embeddings f_{node}^{depth} on the node depth, such that the learned tree representations include both the node relationships and the node hierarchy. The node input node features f^{node} are constructed by the concatenation of f_{sen}^{node} and f_{depth}^{node}.

Therefore, we can compute the attention coefficients as below

$$e_{ij}^{node} = f_i^{node} f_j^{node^T}, \tag{16}$$

where we use the matrix multiplication to measure the relationships between the node features (f_i^{node}, f_j^{node}). Eq (12) is further adopted to give the attentional scores α_{ij}. With α_{ij}, we can formulate F_{tree} as

$$\mathrm{F}_{tree} = \sigma\left(\sum_{j \in \mathrm{N}_i} \alpha_{ij} f_j^{node}\right), \tag{17}$$

where N_i denotes the neighborhood of node i, and σ is the nonlinear activation used in GAT.

3.5.4 Retrieval Training

The recipe representations F_{rec} is obtained from the concatenation of the ingredient features F_{ing}, instruction features

imited to: CENTRE FOR DEVELOPMENT OF ADVANCED COMPUTING - CDAC - Thiruvananthapuram & Kochi. Downloaded on September 06,2024 at 08:11:19 UTC from IEEE Xp

F_{ins} and recipe tree structure representation F_{tree}. We utilize the triplet loss to train image-to-recipe retrieval model, the objective function is

$$L_{tri} = \sum \Big[d(\mathrm{F}^a_{img}, \mathrm{F}^p_{rec}) - d(\mathrm{F}^a_{img}, \mathrm{F}^n_{rec}) + m \Big] + \sum \Big[d(\mathrm{F}^a_{rec}, \mathrm{F}^p_{img}) - d(\mathrm{F}^a_{rec}, \mathrm{F}^n_{img}) + m \Big], \quad (18)$$

where $d(\bullet)$ denotes the euclidean distance, superscripts a, p and n refer to anchor, positive and negative samples respectively and m is the margin. We follow the practice of previous works [20], [21] to use the *BatchHard* idea proposed in [40], to improve the training effectiveness. Specifically, we dynamically construct the triplets during training phase. In a mini-batch, we select the most distant positive instance and the closest negative instance, given an anchor sample.

4 Experiments

4.1 Dataset and Evaluation Metrics

We evaluate our proposed method on Recipe1M dataset [2], which is one of the largest collection of cooking recipe data with food images. Recipe1M has rich food related information, including the food images, ingredients and cooking instructions. In Recipe1M, there are 252,547, 54,255 and 54,506 food data samples for training, validation and test respectively. These recipe data is collected from some public websites, which are uploaded by users.

For the recipe generation task, we evaluate the model using the same metrics as prior works [1], [13]: perplexity, BLEU [41] and ROUGE [42]. Perplexity is used in [1], it measures how well the learned word probability distribution matches the target recipes. BLEU is computed based on the average of unigram, bigram, trigram and 4-gram precision. We use ROUGE-L to test the longest common subsequence. ROUGE-L is a modification of BLEU, where ROUGE-L score metric is measuring recall instead of precision. Therefore, we can use ROUGE-L to measure the fluency of generated recipes.

Regrading the image-to-recipe retrieval task, we evaluate our proposed framework as the common practice used in prior works [2], [20], [22], [23], [26]. To be specific, median retrieval rank (MedR) and recall at top K (R@K) are used. MedR measures the median rank position among where true positives are returned. Therefore, higher performance comes with a lower MedR score. Given a food image, R@K calculates the fraction of times that the correct recipe is found within the top-K retrieved candidates, and vice versa. Different from MedR, the performance is directly proportional to the score of R@K. In the test phase, we first sample 10 different subsets of 1,000 pairs (1 k setup), and 10 different subsets of 10,000 (10 k setup) pairs. It is the same setting as in [2]. We then consider each item from food image modality in subset as a query, and rank samples from recipe modality according to L2 distance between the embedding of image and that of recipe, which is served as image-to-recipe retrieval, and vice versa for recipe-to-image retrieval.

4.2 Implementation Details

We adopt a 3-layer ON-LSTM [11] to output the sentence-level tree structure, taking about 50 epoch training to get converged. We set the learning rate as 1, batch size as 60, and the input embedding size is 400, which is the same as original work [11]. We select recipes containing over 4 sentences in Recipe1M dataset for training. And we randomly select several consecutive sentences as the context and the following one as the correct one. We set K as 3. We show some of the predicted sentence-level tree structures for recipes in Fig. 6.

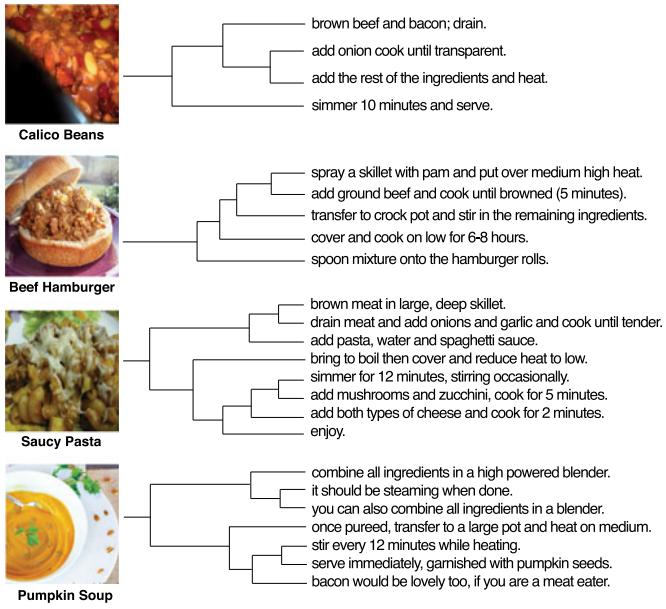


Fig. 6. The visualization of predicted sentence-level trees for recipes. The latent tree structure is obtained from unsupervised learning. The results indicate that we can get reasonable parsing tree structures with varying recipe length.

We use two different ingredient encoders in the experiments, i.e., the non-pretrained and pretrained language model. Using non-pretrained model is to compare with the prior work [1], where they use a word embedding layer to give the ingredient embeddings. We use BERT [35] as the pretrained language model, giving 512-dimensional features. The image encoder is used with a ResNet-50 [45] pretrained on ImageNet [46]. And we map image output features to the dimension of 512, to align with the ingredient features. We adopt a RNN for tree adjacency sequence generation, where the RNN initial hidden state is initialized as the previous image features. The RNN layer is set as 2 and the hidden state size is 512. The tree embedding model is graph attention network (GAT), its attention head number is set as 6. The output tree feature dimension is set the same as that of image features. We use the same settings in language decoder as prior work [1], a 16-layer transformer [37]. The number of attention heads in the decoder is set as 8. We use greedy search during text generation, and the maximum generated instruction length is 150. We set λ_1 and λ_2 in Eq. (1) as 1 and 0.5 respectively. The model is trained using Adam [47] optimizer with the batch size of 16. Initial learning rate is set as 0.001, which decays 0.99 each epoch. The BERT model finetune learning rate is 0.0004.

In the retrieval model training, we use the pretrained ResNet-50 model to give image features. We then adopt an one-layer bi-directional LSTM with the self-attention mechanism [37] and a 4-layer transformer respectively to encode

imited to: CENTRE FOR DEVELOPMENT OF ADVANCED COMPUTING - CDAC - Thiruvananthapuram & Kochi. Downloaded on September 06,2024 at 08:11:19 UTC from IEEE Xp

TABLE 1
Food Cross-Modal Retrieval Evaluation

Size of Test Set	Methods	Image-to-Recipe Retrieval				Recipe-to-Image Retrieval			
		medR ↓	R@1 ↑	R@5 ↑	R@10 ↑	medR ↓	R@1 ↑	R@5 ↑	R@10 ↑
1 k	CCA [43]	15.7	14.0	32.0	43.0	24.8	9.0	24.0	35.0
	SAN [44]	16.1	12.5	31.1	42.3	-	-	-	-
	JE [2]	5.2	24.0	51.0	65.0	5.1	25.0	52.0	65.0
	AM [22]	4.6	25.6	53.7	66.9	4.6	25.7	53.9	67.1
	AdaMine [23]	**1.0**	39.8	69.0	77.4	**1.0**	40.2	68.1	78.7
	R^2GAN [26]	**1.0**	39.1	71.0	81.7	**1.0**	40.6	72.6	83.3
	ACME [20]	**1.0**	51.8	80.2	87.5	**1.0**	52.8	80.2	87.6
	Ours	**1.0**	**53.5**	**81.5**	**88.8**	**1.0**	**55.0**	**82.0**	**88.8**
10 k	JE [2]	41.9	-	-	-	39.2	-	-	-
	AM [22]	39.8	7.2	19.2	27.6	38.1	7.0	19.4	27.8
	AdaMine [23]	13.2	14.9	35.3	45.2	12.2	14.8	34.6	46.1
	R^2GAN [26]	13.9	13.5	33.5	44.9	11.6	14.2	35.0	46.8
	ACME [20]	6.7	22.9	46.8	57.9	6.0	24.4	47.9	59.0
	Ours	**6.0**	**23.4**	**48.8**	**60.1**	**5.6**	**24.6**	**50.0**	**61.0**

We show results of baseline models and our proposed method. The models are evaluated on the basis of MedR (lower is better), and R@K (higher is better).

the recipes, to show the difference between using LSTM and transformer in the food retrieval task. An 8-head GAT is used to encode the sentence-level tree structures to give the tree features, which are concatenated with the recipe features. We map the image and recipe features to a common space to do the retrieval training, with the feature size of 1024. We set the batch size and learning rate as 64 and 0.0001 respectively. We decrease the learning rate 0.1 in the 30th epoch.

In terms of the computation cost, we observe our introduced components do not add too much burden to the original model. We use a single V100 for all experiments. The recipe2tree module is shared by the recipe generation and cross-modal retrieval models, where its training phase costs about 4 hours and inference time is about 0.01 s for each sample. For the recipe generation model, the baseline model [35] takes around 1.5 hours per epoch for training and 0.04 s per sample for inference; the baseline model [35] with structural representations takes around 1.7 hours per epoch for training and 0.05 s per sample for inference. For the food cross-modal retrieval model, the baseline model (LSTM) takes around 0.5 h per epoch for training and 0.02 s per sample for inference; the baseline model (LSTM) using structural representations takes around 0.6 h per epoch for training and 0.02 s per sample for inference.

4.3 Baselines

4.3.1 Recipe Generation

Since Recipe1M has different data components from standard MS-COCO dataset [10], it is hard to implement some prior image captioning models in Recipe1M. To the best of our knowledge, [1] is the only recipe generation work on Recipe1M dataset, where they use the Encoder-Decoder architecture. Based on the ingredient and image features, they generate the recipes with transformer [37].

The SGN model we proposed is an extension of the baseline model, which learns the sentence-level tree structure of target recipes by an unsupervised approach. We infer the tree structures of recipes before language generation, adding an additional module on the baseline model. It means that our proposed SGN can be applied to many other deep model architectures and vision-language datasets. We test the performance of SGN with two ingredient encoders, 1) non-pretrained word embedding model and 2) pretrained BERT model. Word embedding model is used in [1], trained from scratch. BERT model [35] is served as another baseline, to test if SGN can improve language generation performance further under a powerful encoder. We use ResNet-50 in both two baseline models. The recipe generation results are shown in Table 2.

4.3.2 Food Cross-Modal Retrieval

Canonical Correlation Analysis (CCA) [43] is one of the most widely-used classic models for learning a common embedding from different feature spaces, which learns linear projections for images and text to maximize their feature correlation. Salvador *et al.* [2] aim to learn joint embeddings (JE) for images and recipes, where they adopt cosine loss to align image-recipe pairs and classification loss to regularize the learning. In SAN [44] and AM [22], they introduce attention mechanism to different levels of recipes including food titles, ingredients and cooking instructions. AdaMine [23] is an adaptive learning schema in the training phase, helping the model perform an adaptive mining for significant triplets. Later, adversarial methods [20], [26] are proposed for retrieval alignment. Specifically, Zhu *et al.* [26] use two-level ranking loss at embedding and image spaces in R^2GAN. ACME [20] introduce the translation consistency component to allow feature distributions from different modalities to be similar. The evaluation results are shown in Table 1.

4.4 Evaluation Results

4.4.1 Recipe Generation

Language Generation Performance. We show the performance of SGN for recipe generation against the baselines in Table 2. In both baseline settings, our proposed method SGN outperforms the baselines across all metrics. In the method of

imited to: CENTRE FOR DEVELOPMENT OF ADVANCED COMPUTING - CDAC - Thiruvananthapuram & Kochi. Downloaded on September 06,2024 at 08:11:19 UTC from IEEE Xp

TABLE 2
Recipe Generation Evaluation

Methods	Perplexity ↓	BLEU ↑	ROUGE-L ↑
Non-pretrained Model [1]	8.06	7.23	31.8
+ SGN	7.46	9.09	33.4
TIRG [24]	7.83	7.95	32.4
+ SGN	7.56	9.24	34.5
VAL [25]	7.61	8.83	34.2
+ SGN	7.05	10.43	35.6
Pretrained Model [35]	7.52	9.29	34.8
- ingr	8.16	3.72	31.0
- image	7.62	5.74	32.1
+ SGN	**6.67**	**12.75**	**36.9**

We give results of baseline models without and with the proposed SGN for comparison. We also present ablative results of the pretrained model performance without the image and ingredient (ingr) features respectively. The model is evaluated with perplexity, BLEU and ROUGE-L.

TABLE 3
Generated Recipe Average Length for Recipe Generation

Methods	Recipe Average Length
Pretrained Model [35]	66.9
+ SGN	112.5
Ground Truth (Human)	116.5

We compare the average length between recipes from different sources.

non-pretrained model, SGN achieves a BLEU score more than 9.00, which is about 25% higher than the current state-of-the-art method. Here we directly concatenate the image and text features. To compare the impact of different image-text features fusion methods, we also give results of TIRG [24] and VAL [25]. Specifically, TIRG [24] adopts an LSTM and a ResNet CNN to encode the text and the images respectively, then with the gating and residual connections the image-text fused features can be obtained. In VAL [25], Chen *et al.* also use an LSTM and a ResNet-50 to get the text and image features respectively. They feed the concatenation of the image and text features into the transformer, where the concatenated features are further processed with the attention mechanism to produce the fused features. We observe there is a margin between the performance of TIRG and VAL, since VAL use the composite transformers that are plugged into different convolution layers to compose the vision and language contents, obtaining more fine-grained features than TIRG.

When we shift to the pretrained model method [35], we can see that the pretrained language model gets comparable results as "TIRG + SGN" model. We also show the ablative results of models trained without ingredient (ingr) and image features respectively, where we observe the ingredient features help more on the generation results. When incrementally adding SGN to the pretrained model, the performance of SGN is significantly superior to all the baselines by a substantial margin. Although we only use the concatenation method to fuse the image and text features, we utilize the pretrained BERT model to extract the text features, which gives better results than "VAL + SGN". This may indicate the significance of the pretrained language model. On the whole, the efficacy of SGN is shown to be very promising, outperforming the state-of-the-art method across different metrics consistently.

Impact of Structure Awareness. To explicitly suggest the impact of tree structures on the final recipe generation, we compute the average length for the generated recipes, as shown in Table 3. Average length can reflect the text structure on node numbers. It is observed that SGN generates recipes with the most similar length as the ground truth, indicating the help of the tree structure awareness.

4.4.2 Food Cross-Modal Retrieval

Cross-Modal Retrieval Performance. In Table 1, we compare the results of our proposed method with various state-of-the-art methods against different metrics. Specifically, ACME [20] takes triplet loss and adversarial training to learn image-recipe alignment, which gives superior results over other state-of-the-art models. ACME mainly focuses on improving cross-modal representation consistency at the common space with the cross-modal translation. Here we do not use the adversarial training of ACME and implement the self-attention mechanism on the LSTM to train the retrieval model. We add the tree structure representations on the baseline, it can be observed that we further boost the performance across all the metrics. It suggests that our unsupervisedly-learned tree structures also can be applied on the retrieval task and have positive value to the whole model.

Ablation Study of Different Compositions of Tree Features. In Table 4, we adopt the LSTM as the recipe encoder. We first directly use the projection of the adjacency matrix as the node features, and further construct the tree embeddings with GAT. We then adopt the sentence features f_{node}^{sen} and the concatenation of f_{node}^{sen} and f_{node}^{depth} respectively to be the node features. We can see that the including more information into the tree features helps on improving the food cross-modal retrieval performance.

Ablation Study of Different Recipe Encoders. We show the results of using LSTM and transformer respectively as the recipe embedding backbone in Table 5. We adopt the self-attention based LSTM [37] and transformer to encode the recipes. It can be seen that since the attention mechanism is implemented on the LSTM, the LSTM model achieves similar performance to the transformer model. And the learned tree features can boost the retrieval performance in both settings.

Ablation Study of Different Feature Fusion Methods. Since we use the concatenation of the ingredient features F_{ing}, cooking instruction features F_{ins} and recipe tree structure

TABLE 4
Ablation Studies on the Composition of Tree Features for Cross-Modal Retrieval

Node Features	R@1 ↑	R@5 ↑	R@10 ↑
Adjacency matrix projection	52.7	81.3	88.4
f_{node}^{sen}	53.3	81.5	88.5
f_{node}^{sen} + f_{node}^{depth}	**53.5**	**81.5**	**88.8**

We adopt different tree node embeddings and report the results on rankings of size 1 k, with the basis of R@K (higher is better). Here, we use the LSTM to encode the recipes.

imited to: CENTRE FOR DEVELOPMENT OF ADVANCED COMPUTING - CDAC - Thiruvananthapuram & Kochi. Downloaded on September 06,2024 at 08:11:19 UTC from IEEE Xp

TABLE 5
Ablation Studies of Different Recipe Encoders for Cross-Modal Retrieval, Where the LSTM and Transformer are Used Respectively as the Backbone to Encode the Cooking Recipes

Method	R@1 ↑	R@5 ↑	R@10 ↑
baseline (LSTM)	52.5	81.1	88.4
+ tree (cat)	53.5	81.5	88.8
+ tree (TIRG [24])	54.2	81.9	88.6
+ tree (VAL [25])	54.0	**82.0**	89.0
baseline (transformer)	53.4	81.5	87.8
+ tree (cat)	54.3	81.6	88.4
+ tree (TIRG [24])	**54.7**	81.9	**89.1**
+ tree (VAL [25])	54.4	81.8	88.7

We experiment with various methods to fuse the tree and recipe features, where ***cat*** *represents the concatenation method, TIRG [24] and VAL [25] are the existing cross-modal feature learning methods. The results are reported on rankings of size* $1\ k$*, with the basis of R@K (higher is better).*

TABLE 6
Ablation Studies of Incorporating the img2tree Module Into the Cross-Modal Retrieval Model

Method	coefficient λ	R@1 ↑	R@5 ↑	R@10 ↑
recipe2tree only		**53.5**	**81.5**	**88.8**
+ img2tree	1	52.5	81.1	88.4
+ img2tree	0.1	53.4	81.4	88.5
+ img2tree	0.01	52.9	81.3	88.4

recipe2tree *indicates using the tree structure for recipe features only.* img2tree *represents we also learn to generate the tree structures from the images during the retrieval training phase. The* coefficient λ *indicates trade-off parameter on the tree generation loss* L_{tree}*. The results are reported on rankings of size* $1\ k$*, with the basis of R@K (higher is better).*

features F_{tree} as the recipe representations, here we also adopt TIRG [24] and VAL [25] respectively to replace the concatenation method and produce the fused features of F_{ing}, F_{ins} and F_{tree}. The experiment results are shown in Table 5. Technically, TIRG [24] utilized the gating and residual connections to obtain the cross-modal fused features. VAL [25] applied spatial and channel attention transform on the image-text features. Note that here we do not fuse the image and text features together, but the tree and text features instead. That means we cannot apply the spatial self-attention transformation on the tree-text features as the original implementation in VAL [25]. Therefore, we only conduct the channel attention of VAL [25]. The results show TIRG [24] gives more improvements over the baseline than VAL [25]. We also observe that adding tree features or using better feature fusion methods mainly brings improvements on the R@1, which means better top-ranking results can be obtained.

Ablation Study of Using the img2tree Module. In Table 6, we aim to evaluate the performance of incorporating the img2tree module into the retrieval framework. Specifically, we learn to generate the trees from food images, which is supervised by the pseudo ground truth from the recipe2tree module. train the model with loss of $L_{tri} + \lambda L_{tree}$, where λ is the trade-off parameter. The inferred tree structures are also encoded with the GAT and concatenated with the food image features, which are used as the query to retrieve the corresponding recipes. We conduct experiments with various λ. However, we observe there is no performance improvement of adding the img2tree module on the image feature learning, which may indicate the differences between the recipe generation and food cross-modal retrieval tasks. Technically, in the retrieval setting, we attempt to do precise matching between the image and recipe features, where the image features contain the generated tree representations and recipe features contain the pseudo ground truth tree representations. If the predicted tree structures from the images are not accurate enough, i.e., fail to produce the same node numbers and the same node relationships as the pseudo ground truth, then the tree embeddings will also be different, which decreases the retrieval performance. While in the recipe generation task, the extra guidance of plausible tree structures can help the model capture the preliminary cooking procedure, thus giving better performance than the model without inferred structures. Nevertheless, better results can also be expected if the prediction performance of img2tree module can be improved further. We leave it for the future work.

4.5 Qualitative Results

4.5.1 Sentence-Level Tree Parsing Results

In Fig. 6, we visualize some parsing tree results of our proposed recipe2tree module. Due to there is no human labelling on the recipe tree structures, we can hardly provide a quantitative analysis on the parsing trees.

We show some examples with varying paragraph length in Fig. 6. The first two rows show the tree structures of relatively short recipes. Take the first row (*calico beans*) as example, the generated tree set the food pre-processing part (step 1) as a separate leaf node, and two main cooking steps (step 2&3) are set as deeper level nodes. The last *simmer* step is conditioned on previous three steps, which is put in another different tree level. We can see that the parsing tree results correspond with common sense and human experience.

In the last two rows of Fig. 6, we show the parsing results of recipes having more than 5 sentences. The tree of *pumpkin soup* indicates clearly two main cooking phases, i.e., before and after ingredient pureeing. Generally, the proposed recipe2tree generated sentence-level parsing trees look plausible, helping on the inference for recipe generation.

4.5.2 Recipe Generation Results

We present some recipe generation results in Fig. 7. We consider three types of recipe sources, the human, models trained without and with SGN. Each recipe accompanies with a food image. We can observe that recipes generated by model with SGN have similar length with that written by users. It may indicate that, instead of generating language directly from the image features, allowing the deep model to be aware of the structure first brings benefits for the following recipe generation task.

We indicate the matching parts between recipes provided by users and that generated by models, in red words. It is observed that SGN model can produce more coherent and detailed recipes than non-SGN model. For example, in the middle column of Fig. 7, SGN generated recipes include some ingredients that do not exist in the non-SGN generation, but are contained in users' recipes, such as *onion*, *lettuce* and *tomato*.

imited to: CENTRE FOR DEVELOPMENT OF ADVANCED COMPUTING - CDAC - Thiruvananthapuram & Kochi. Downloaded on September 06,2024 at 08:11:19 UTC from IEEE Xp

User	In a greased oval 5-6 quart slow cooker, combine the vegetables and broth. Place drumsticks over vegetables. Sprinkle remaining ingredients over all. Cover and cook on low for 5-5 1/2 hours, or until meat thermometer registers 180° f.	In medium bowl, gently mix together chicken, bread crumbs, egg, milk, lemon juice, mint, oregano, salt and pepper until well combined. Shape into 4 patties each about 3/4 inch thick. In a large nonstick skillet heat oil over medium-high heat. Cook patties about 8 minutes turning once, until golden brown and no longer pink inside. Meanwhile, cut pitas in half crosswise to make 4 pockets. Warm pitas in 300 degree oven wrapped in aluminum foil for 5 minutes or in the microwave for 1 minute. Spread inside of pita with mayo then place cooked patty inside with a slice of red onion, tomato, and some lettuce. Enjoy.	Bring a medium size pot of water to a boil, add rice. Bring back to a boil, then reduce heat to simmer. Let rice simmer 15-20 minutes, until tender. Place beans and rice in a medium size saucepan. Heat over a medium heat, stirring frequently. Stir in reserved bean liquid as needed. Remove pan from heat and stir in lemon juice, garlic powder and cilantro. Let sit a moment, and stir in fresh oregano. Serve immediately.
Model without SGN	Place all ingredients in crock pot. Cook on low for 8 hours.	In a large bowl, combine the ground chicken, bread crumbs, egg, milk, lemon juice, oregano, mint, salt and pepper. Mix well. Shape into 4 patties. In a large skillet, heat oil over medium heat. Cook patties for 5 minutes on each side or until cooked through. Serve with lemon wedges.	In a medium saucepan, bring water to a boil. Stir in rice, reduce heat to low, cover, and simmer until rice is tender, about 20 minutes. Stir in black beans, lemon juice, garlic powder, and cilantro.
Model with SGN	In a large pot, combine potatoes, carrots, celery, onion, and garlic. Add chicken broth, salt, pepper, and thyme. Bring to a boil, reduce heat and simmer for 20 minutes. Add potatoes and simmer for another 10 minutes. Add cream and simmer for another 10 minutes. Serve with crusty bread. Enjoy!	Combine chicken, bread crumbs, egg, milk, lemon juice, oregano, mint, salt and pepper in a large bowl. Shape into 4 patties. Heat oil in a large nonstick skillet over medium-high heat. Cook patties until golden brown, about 5 minutes per side. Transfer to a plate. Add tomato slices to skillet and cook until lightly browned, about 1 minute. Add onion and cook until tender, about 2 minutes. Stir in tomato sauce and cook until heated through, about 1 minute. Serve with lettuce, tomato slices, and burgers. Note: you can substitute any combination of the tomato slices, onion, and garlic.	Bring rice and water to a boil in a saucepan. Reduce heat to low, cover, and simmer until rice is tender, 20 to 25 minutes. Stir black beans, lemon juice, garlic powder, and cilantro into rice; cook until heated through, about 5 more minutes. Serve warm. Enjoy!

Fig. 7. Visualization of recipes from different sources. We show the food images and the corresponding recipes, obtained from users and different types of models. Words in red indicate the matching parts between recipes uploaded by users and that generated by models. Words in yellow background show the redundant generated sentences.

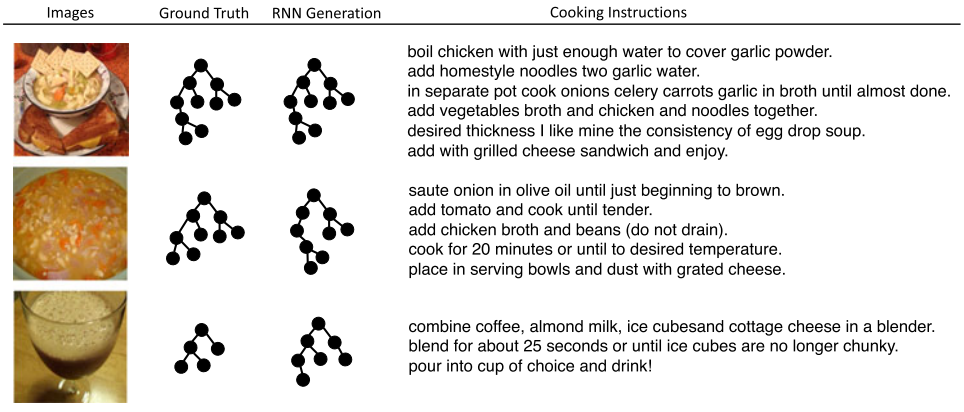


Fig. 8. The comparison between the pseudo ground truth trees (produced by recipe2tree module) and img2tree generated tree structures.

However, although SGN can generate longer recipes than non-SGN model, it may produce some redundant sentences. These useless sentences are marked with yellow background, as shown in the first column of Fig. 7. Since *Cream* is not supposed to be used in the chicken soup, in the future work, we may need to use the input ingredient information better to guide the recipe generation.

4.5.3 Tree Generation Results

There are some graph evaluation metrics proposed in [34], however, these metrics are used for unconditional graph evaluation. How to evaluate the graph similarities for conditional generation remains an open problem. Here we show some examples of generated recipe tree structures in Fig. 8 for qualitative analysis. Tree generation results from image features are by-product of our proposed SGN framework. They are used to improve the final recipe generation performance.

It is notable that only the leaf nodes in the tree represent the sentences of recipe. We can observe that the overall img2tree generated structures look similar with the ground truth trees, which are produced by recipe2tree module. And the generated trees have some diversity. However, it is hard to align the number of generated nodes with the ground truth. For example, in the last row of Fig. 8, the generated tree has one more node than the ground truth. Nevertheless, the generated trees along with the tree embedding network can improve the final recipe generation performance based on the quantitative results.

4.5.4 Food Cross-Modal Retrieval Results

We show the visualizations of recipe-to-image and image-to-recipe retrieval in Figs. 9 and 10 respectively, where we apply 1 k setup. In Fig. 9, we conduct the recipe-to-image retrieval. Specifically, we also select three different recipes and take them as queries, the right column lists the top-3 retrieved results. The results demonstrate that our model not only has good performance on cross-modal retrieval, and also helps to discover similar samples from the same modality. For example, in the top row all of our top-3 ranked food images contain *chicken*, which share semantic similarity. In Fig. 10, we randomly select images from three different food classes, i.e., *potato cake*, *salad* and *chicken wellingtons*. Based on the image queries, we can see that our model retrieves the cooking recipes across various food classes correctly. This also indicates some potential applications, for example, the

imited to: CENTRE FOR DEVELOPMENT OF ADVANCED COMPUTING - CDAC - Thiruvananthapuram & Kochi. Downloaded on September 06,2024 at 08:11:19 UTC from IEEE Xp

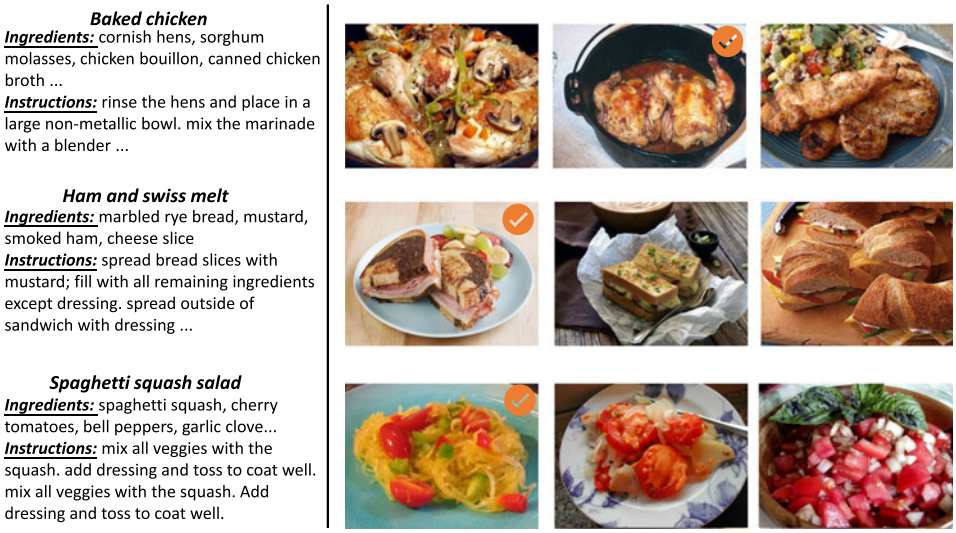


Fig. 9. The visualization of recipe-to-image retrieval results. For each recipe query, we show the top-3 retrieved food images, where the yellow ticks suggest the correctly retrieved samples.

model can automatically produce the corresponding cooking recipes given users' uploaded food images.

5 Conclusion

In this paper, we have proposed a novel unsupervised learning approach to generate the sentence-level tree structures for the cooking recipes, which can benefit the recipe generation and food cross-modal retrieval tasks. To be specific, we propose effective ways to address some challenging problems, including unsupervisedly extracting the paragraph structures, generating tree structures from images and using the tree structures for recipe generation and food cross-modal retrieval models. Technically, we extend ON-LSTM to label recipe tree structures using an unsupervised manner.

Fig. 10. The visualization of image-to-recipe retrieval results. The top row indicates the image query, the bottom row shows the corresponding retrieved recipes, which are correctly matched with the ground truth.

In the recipe generation model, we propose to use RNN to generate the tree structures from food images, and adopt the inferred trees to enhance the generation. In the cross-modal retrieval model, we integrate the tree structural information into the cooking recipe representations. We demonstrate our learned tree structures improve both of the recipe generation and cross-modal model performance. We have conducted quantitative and qualitative analysis for both tasks, the results show that models with our proposed tree structures outperform various baseline models.

However, since the food image semantic structures can hardly be obtained, we attempt to learn the structural information for text only in our proposed framework, while we fail to build the inner semantic relationships for visual data. In the future work, we can try to construct the structural representations for the visual data. We may extend our current work to cooking video datasets, where the relationships between video frames and the correspondence between recipes and frames can be built.

Acknowledgments

Any opinions, findings and conclusions or recommendations expressed in this material are those of the authors and do not reflect the views of National Research Foundation, Singapore.

References

[1] A. Salvador, M. Drozdzal, X. Giro-i Nieto, and A. Romero, "Inverse cooking: Recipe generation from food images," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 10453–10462.
[2] A. Salvador *et al.*, "Learning cross-modal embeddings for cooking recipes and food images," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 3020–3028.
[3] K. Wang, Q. Yin, W. Wang, S. Wu, and L. Wang, "A comprehensive survey on cross-modal retrieval," 2016, *arXiv:1607.06215*.
[4] L. Chen, H. Zhang, J. Xiao, X. He, S. Pu, and S.-F. Chang, "Counterfactual critic multi-agent training for scene graph generation," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2019, pp. 4613–4623.
[5] O. Vinyals, A. Toshev, S. Bengio, and D. Erhan, "Show and tell: A neural image caption generator," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 3156–3164.

imited to: CENTRE FOR DEVELOPMENT OF ADVANCED COMPUTING - CDAC - Thiruvananthapuram & Kochi. Downloaded on September 06,2024 at 08:11:19 UTC from IEEE Xp

[6] S. Chun, S. J. Oh, R. S. De Rezende, Y. Kalantidis, and D. Larlus, "Probabilistic embeddings for cross-modal retrieval," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 8415–8424.

[7] J. Gu, S. Joty, J. Cai, H. Zhao, X. Yang, and G. Wang, "Unpaired image captioning via scene graph alignments," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2019, pp. 10323–10332.

[8] J. Gu, J. Cai, S. R. Joty, L. Niu, and G. Wang, "Look, imagine and match: Improving textual-visual cross-modal retrieval with generative models," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 7181–7189.

[9] B. A. Plummer, L. Wang, C. M. Cervantes, J. C. Caicedo, J. Hockenmaier, and S. Lazebnik, "Flickr30 k entities: Collecting region-to-phrase correspondences for richer image-to-sentence models," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2015, pp. 2641–2649.

[10] X. Chen *et al.*, "Microsoft COCO captions: Data collection and evaluation server," 2015, *arXiv:1504.00325*.

[11] Y. Shen, S. Tan, A. Sordoni, and A. Courville, "Ordered neurons: Integrating tree structures into recurrent neural networks," 2018, *arXiv:1810.09536*.

[12] L. Logeswaran and H. Lee, "An efficient framework for learning sentence representations," 2018, *arXiv:1803.02893*.

[13] H. Wang, G. Lin, S. C. Hoi, and C. Miao, "Structure-aware generation network for recipe generation from images," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 359–374.

[14] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," 2014, *arXiv:1409.0473*.

[15] D. Golland, P. Liang, and D. Klein, "A game-theoretic approach to generating spatial descriptions," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2010, pp. 410–419.

[16] S. Kazemzadeh, V. Ordonez, M. Matten, and T. Berg, "Referitgame: Referring to objects in photographs of natural scenes," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2014, pp. 787–798.

[17] M. Z. Hossain, F. Sohel, M. F. Shiratuddin, and H. Laga, "A comprehensive survey of deep learning for image captioning," *ACM Comput. Surv.*, vol. 51, no. 6, pp. 1–36, 2019.

[18] W. Min, S. Jiang, L. Liu, Y. Rui, and R. Jain, "A survey on food computing," *ACM Comput. Surv.*, vol. 52, no. 5, pp. 1–36, 2019.

[19] L. Bossard, M. Guillaumin, and L. Van Gool, "Food-101–mining discriminative components with random forests," in *Proc. Eur. Conf. Comput. Vis.*, 2014, pp. 446–461.

[20] H. Wang, D. Sahoo, C. Liu, E.-P. Lim, and S. C. Hoi, "Learning cross-modal embeddings with adversarial networks for cooking recipes and food images," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 11572–11581.

[21] H. Wang *et al.*, "Cross-modal food retrieval: Learning a joint embedding of food images and recipes with semantic consistency and attention mechanism," *IEEE Trans. Multimedia*, vol. 24, pp. 2515–2525, 2021.

[22] J.-J. Chen, C.-W. Ngo, F.-L. Feng, and T.-S. Chua, "Deep understanding of cooking procedure for cross-modal recipe retrieval," in *Proc. ACM Multimedia Conf. Multimedia Conf.*, 2018, pp. 1020–1028.

[23] M. Carvalho, R. Cadène, D. Picard, L. Soulier, N. Thome, and M. Cord, "Cross-modal retrieval in the cooking context: Learning semantic text-image embeddings," in *Proc. 41st Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2018, pp. 35–44.

[24] N. Vo *et al.*, "Composing text and image for image retrieval-an empirical odyssey," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 6439–6448.

[25] Y. Chen, S. Gong, and L. Bazzani, "Image search with text feedback by visiolinguistic attention learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 3001–3011.

[26] B. Zhu, C.-W. Ngo, J. Chen, and Y. Hao, "R2GAN: Cross-modal recipe retrieval with generative adversarial network," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 11477–11486.

[27] S. Wu, D. Zhang, N. Yang, M. Li, and M. Zhou, "Sequence-to-dependency neural machine translation," in *Proc. 55th Annu. Meeting Assoc. Comput. Linguistics*, 2017, pp. 698–707.

[28] R. Socher *et al.*, "Recursive deep models for semantic compositionality over a sentiment treebank," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2013, pp. 1631–1642.

[29] A. Williams, N. Nangia, and S. R. Bowman, "A broad-coverage challenge corpus for sentence understanding through inference," 2017, *arXiv:1704.05426*.

[30] Y. Shen, Z. Lin, C.-W. Huang, and A. Courville, "Neural language modeling by jointly learning syntax and lexicon," 2017, *arXiv:1711.02013*.

[31] Y. Jia, Y. Ye, Y. Feng, Y. Lai, R. Yan, and D. Zhao, "Modeling discourse cohesion for discourse parsing via memory network," in *Proc. 56th Annu. Meeting Assoc. Comput. Linguistics*, 2018, pp. 438–443.

[32] X. Lin, S. Joty, P. Jwalapuram, and S. Bari, "A unified linear-time framework for sentence-level discourse parsing," 2019, *arXiv:1905.05682*.

[33] Y. Li, O. Vinyals, C. Dyer, R. Pascanu, and P. Battaglia, "Learning deep generative models of graphs," 2018, *arXiv:1803.03324*.

[34] J. You, R. Ying, X. Ren, W. L. Hamilton, and J. Leskovec, "GraphRNN: Generating realistic graphs with deep auto-regressive models," 2018, *arXiv:1802.08773*.

[35] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.

[36] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," 2017, *arXiv:1710.10903*.

[37] A. Vaswani *et al.*, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.

[38] D. Britz, A. Goldie, M.-T. Luong, and Q. Le, "Massive exploration of neural machine translation architectures," 2017, *arXiv:1703.03906*.

[39] R. Kiros *et al.*, "Skip-thought vectors," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 3294–3302.

[40] A. Hermans, L. Beyer, and B. Leibe, "In defense of the triplet loss for person re-identification," 2017, *arXiv:1703.07737*.

[41] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: A method for automatic evaluation of machine translation," in *Proc. 40th Annu. Meeting Assoc. Comput. Linguistics*, 2002, pp. 311–318.

[42] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Proc. Text Summarization Branches Out*, 2004, pp. 74–81.

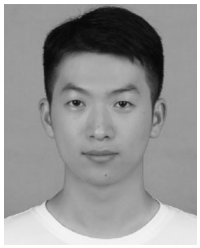
[43] H. Hotelling, "Relations between two sets of variates," *Biometrika*, vol. 28, no. 3/4, pp. 321–377, 1936.

[44] J. Chen, L. Pang, and C.-W. Ngo, "Cross-modal recipe retrieval: How to cook this dish?," in *Proc. Int. Conf. Multimedia Model.*, 2017, pp. 588–600.

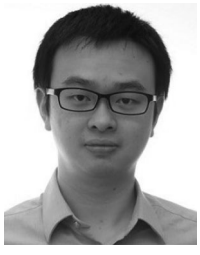
[45] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.

[46] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2009, pp. 248–255.

[47] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.



Hao Wang is currently working toward the PhD degree with the School of Computer Science and Engineering, Nanyang Technological University, Singapore. His research interests include cross-modal generation and computer vision.



Guosheng Lin is currently an assistant professor with the School of Computer Science and Engineering, Nanyang Technological University, Singapore. His research interests include computer vision and machine learning.

imited to: CENTRE FOR DEVELOPMENT OF ADVANCED COMPUTING - CDAC - Thiruvananthapuram & Kochi. Downloaded on September 06,2024 at 08:11:19 UTC from IEEE Xp

Steven C. H. Hoi (Fellow, IEEE) received the bachelor's degree from Tsinghua University, China, in 2002, and the PhD degree in computer science and engineering from The Chinese University of Hong Kong, in 2006. He is currently the managing director of Salesforce Research Asia, and a professor of information systems with Singapore Management University, Singapore. He has served as the editor-in-chief for *Neurocomputing Journal*, general co-chair for ACM SIGMM Workshops on Social Media, program co-chair for the fourth Asian Conference on Machine Learning, book editor for "Social Media Modeling and Computing," guest editor for the *ACM Transactions on Intelligent Systems and Technology*. He is also an ACM distinguished member.

Chunyan Miao received the PhD degree from NTU and was a postdoctoral fellow/instructor with the School of Computing, Simon Fraser University (SFU), Canada. She is currently the chair of the School of Computer Science and Engineering, Nanyang Technological University (NTU), Singapore. She is also a full professor with the Division of Information Systems and director of the Joint NTU-UBC Research Centre of Excellence in Active Living for the Elderly (LILY), School of Computer Engineering, Nanyang Technological University (NTU), Singapore. She visited Harvard and MIT, as a Tan Chin Tuan fellow, collaborating on a large NSF funded research program in social networks and virtual worlds. She has been an adjunct Associate Professor/Associate Professor/founding faculty member with the Center for Digital Media which is jointly managed by The University of British Columbia (UBC) and SFU. She is the editor-in-chief of the *International Journal of Information Technology* published by the Singapore Computer Society.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/csdl.

imited to: CENTRE FOR DEVELOPMENT OF ADVANCED COMPUTING - CDAC - Thiruvananthapuram & Kochi. Downloaded on September 06,2024 at 08:11:19 UTC from IEEE Xp